Linear Regression Using R

From Basics to Predictions

Your Name | Course | Date

What is Regression?

- Regression is a method to predict a numerical outcome (Y) using one or more input variables (X).
- Example questions regression can answer:

  1. How does study time affect exam scores?

  2, How does advertising budget affect sales?

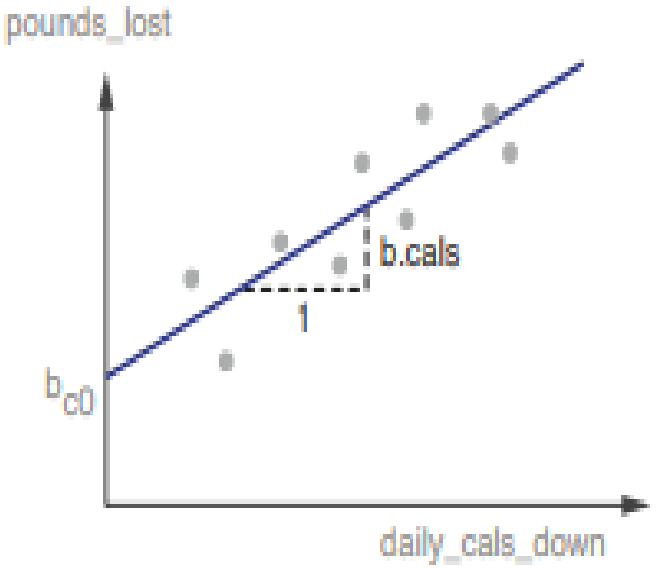
  3. How does age, education, and experience affect income?
- Unlike classification (Yes/No outcomes), regression deals with continuous numbers.

Why Linear Regression?

- Simple and powerful – one of the oldest and most used prediction methods.
- Provides not just predictions, but also insights into:
  1. Which variables matter most?
  2. How strongly do they affect the outcome?
- Always try linear regression first:
  1. If it works → good enough.
  2. If not → diagnostics will guide you to advanced methods.

Understanding linear regression- Basic Idea of Linear Regression

- Linear regression assumes a straight-line relationship between input(s) and output.
- Example: Predicting weight loss
  1. Input X: Calories reduced per day
  2. Output Y: Pounds lost in a month
  3. Relationship: As calories reduced ↑ → pounds lost ↑ (in a straight-line manner)
- Equation: $Y = b0 + b1X + e$
  - b0 = intercept (starting value)
  - b1 = slope (effect of X)
  - e = error term (random noise)



pounds_lost ~ b.cals * daily_cals_down
(plus an offset)

Figure 7.2 The linear relationship between `daily_cals_down` and `pounds_lost`

The relationship between `daily_exercise` and `pounds_lost` would similarly be a straight line. Suppose that the equation of the line shown in figure 7.2 is

```
pounds_lost = bc0 + b.cals * daily_cals_down
```

Multiple Inputs (Multivariable Regression)

- Outcomes often depend on several factors.
- Example: Predicting pounds lost:
  - X1 = Calories reduced
  - X2 = Hours of exercise
- Model: pounds_lost = b0 + b1*calories + b2*exercise
- Each coefficient shows how much that factor influences Y.

Interpretation:

- For each 1 unit increase in calories reduced → pounds lost increases by b1
- For each extra hour of exercise → pounds lost increases by b2

General Form of Linear Regression

- General model with multiple variables:
- y[i] = b0 + b1*x1 + b2*x2 + ... + bn*xn + e[i]

Terminology:

- Dependent variable (Y): outcome to predict
- Independent variables (X): inputs
- Coefficients (b's): effect of each input- Numbers that measure how much each X affects Y.
- Error (e): difference between actual and predicted

When Linear Regression Works Well

- Works best when:
  - Relationship between X and Y is linear
  - Errors are random, not systematic
- Provides unbiased predictions (correct on average).
- Useful for both prediction and understanding relationships.

When Assumptions Fail

- If true relationship is non-linear:
  - Model underestimates in some ranges
  - Overestimates in others
- Example: fitting y = x^2 with a straight line.
- Linear regression may still give a rough approximation, but errors will not be random.

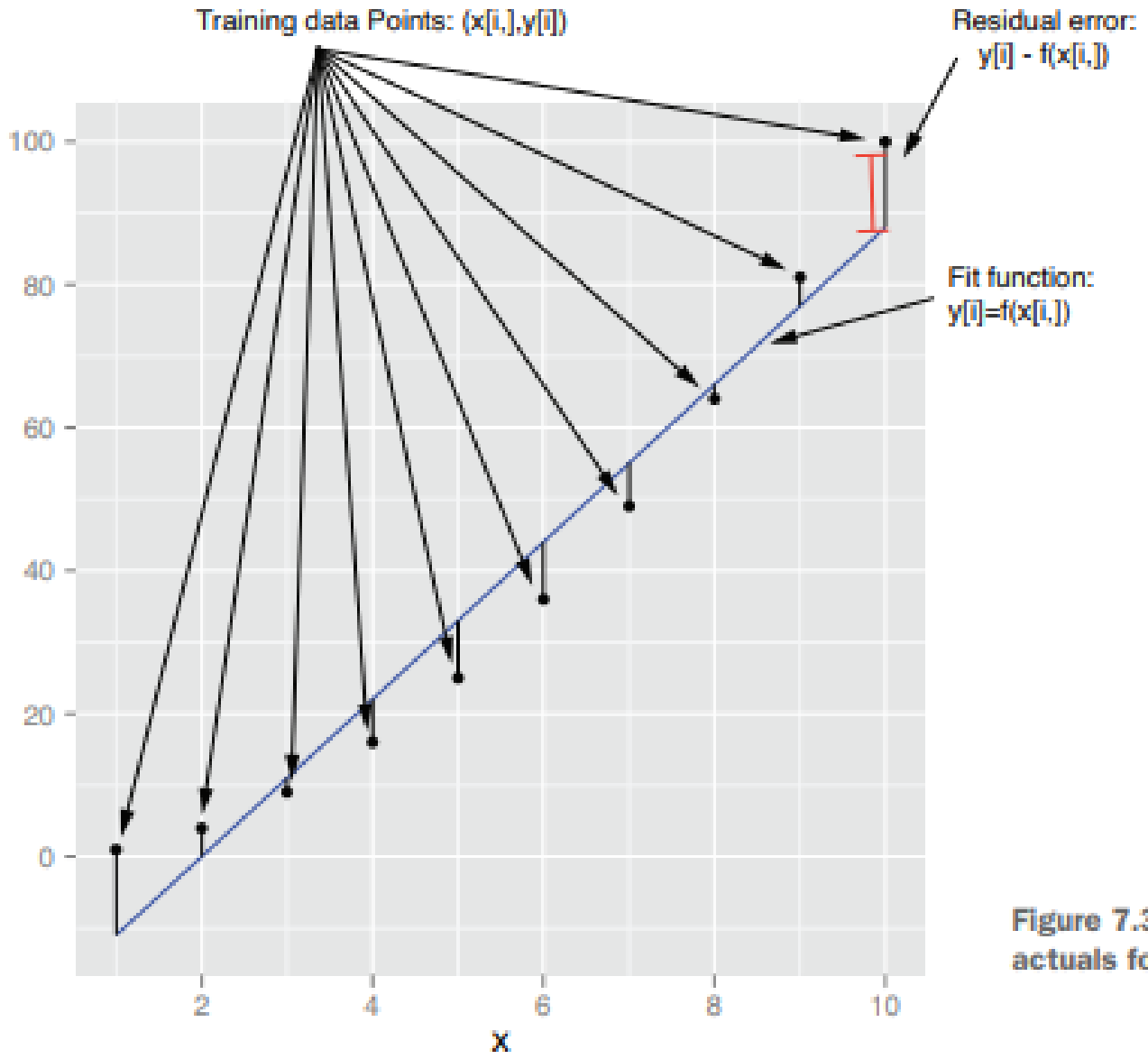


Figure 7.3 Fit versus actuals for $y=x^2$

Explaining Figure 7.3 (Fit vs Actuals for $y = x^2$)

1. **Context:**
   - We try to model $y = x^2$ using a **linear regression line.**
   - This is a case where the true relationship is **non-linear.**
2. **Fitted Line:**
   - Linear regression finds the "best possible" straight line:

$$y \approx -22 + 11x$$

   - This line does not perfectly capture the curve of x^2.
3. **Good Fit in Middle:**
   - In the **training range (x = 1–10)**, the line fits fairly well in the middle region.
   - Predictions are close to actual values around midpoints.
4. **Systematic Errors:**
   - At the edges (small x or large x), the model makes **systematic errors:**
     - **Underestimates** when x is small.
     - **Overestimates** when x is large.
5. **Lesson:**
   - Linear regression can still give a **reasonable approximation** even when data is not truly linear.
   - But errors will show **patterns** (not random).
6. **Takeaway for Practice:**
   - Linear models are simple and useful.
   - But they may not be safe for tasks where the real-world relationship is clearly **non-linear.**

Interpolation vs Extrapolation

- - Interpolation = predicting within training range → safe.
- - Extrapolation = predicting outside observed range → risky.
- - Linear regression often fails when applied beyond observed data.

The PUMS Dataset (US Census)

- PUMS = Public Use Microdata Sample (US Census).
- Contains anonymized personal/household data:
  - Age (AGEP)
  - Sex (SEX)
  - Class of worker (COW)
  - Education (SCHL)
  - Income (PINCP)
- Task: predict personal income (PINCP).

Preparing the Data

- - Restricted to:
  - Full-time employees
  - Age: 20–50 years
  - Income: $1,000 – $250,000
- - Split dataset into:
  - Training set (dtrain) → build model
  - Test set (dtest) → check accuracy

R Code: Building the Model

- psub <- readRDS("psub.RDS")
- set.seed(3454351)
- gp <- runif(nrow(psub))
- dtrain <- subset(psub, gp >= 0.5)
- dtest <- subset(psub, gp < 0.5)
- model <- lm(log10(PINCP) ~ AGEP + SEX + COW + SCHL, data = dtrain)
- dtest$predLogPINCP <- predict(model, newdata = dtest)
- dtrain$predLogPINCP <- predict(model, newdata = dtrain)

- `readRDS("psub.RDS")` → loads the prepared sample of PUMS data.
- Splitting the data:
  - Half for training (to learn the model).
  - Half for testing (to check accuracy).
- Model formula:
  - `lm(log10(PINCP) ~ AGEP + SEX + COW + SCHL, data = dtrain)`
  - Predicts log10 of personal income (PINCP) using:
    - `AGEP` = Age
    - `SEX` = Male/Female
    - `COW` = Class of worker (private, government, etc.)
    - `SCHL` = Education level
- Predictions stored in new columns:
  - `predLogPINCP` in both train and test sets.

Why Log10 of Income?

- - Income varies widely ($1k – $250k).
- - Log transform (log10):
  - Reduces skewness
  - Handles relative errors better
  - Improves stability of predictions
- - Trade-off: predictions slightly underestimate true income.

Building a Linear Regression Model

- -Next step: apply regression on real data.
- - Example: Predicting income using Census data (PUMS).
- - Steps:
- • Load dataset
- • Prepare data (filter, clean)
- • Split into training and testing
- • Fit model using lm() in R
- • Make predictions

Linear Regression Using R

7.1.2 Building a Model & 7.1.3 Making Predictions

Building a Linear Regression Model

- To build a linear regression model in R, we use the function lm() (from the stats package).
- The most important argument in lm() is a formula, written as:-Formula syntax: y ~ x1 + x2 + ... + xn
- This formula tells R:
- y = the dependent variable (what we want to predict).
- x's = independent variables (factors we use for prediction).
- Example: log10(PINCP) ~ AGEP + SEX + COW + SCHL
- This means: predict log10 of income as a function of:

AGEP = Age

SEX = Gender

COW = Class of worker

SCHL = Education level

- Model trained on training dataset.

Understanding lm() in R

- model <- lm(formula, data)
  - model: stores results
  - lm(): builds regression model
  - formula: dependent ~ independent variables
  - data: dataset used to train

- Example:

- model <- lm(log(PINCP, base=10) ~ AGEP + SEX + COW + SCHL, data=dtrain)

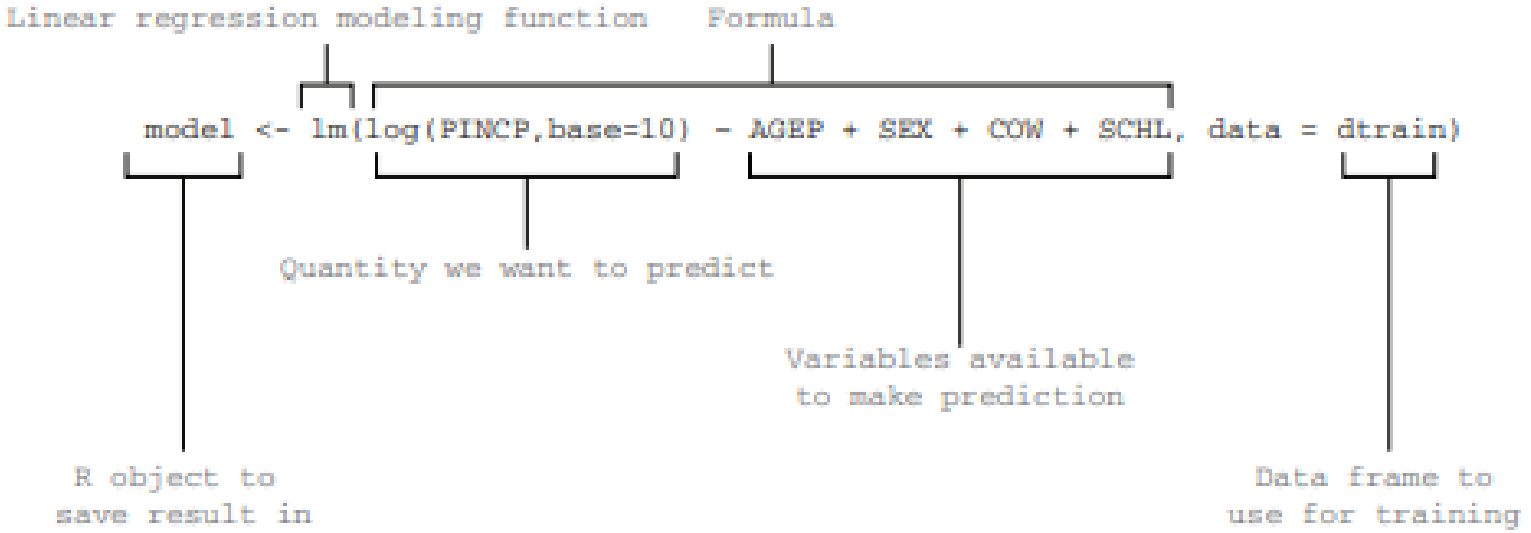


Figure 7.4 Building a linear model using `lm()`

1. **model** → name of the R object where results are stored.
2. **lm()** → function that builds the linear regression model.
3. **Formula (log(PINCP) ~ AGEP + SEX + COW + SCHL):**
   - Left-hand side (before `~`) = dependent variable (*PINCP → personal income*).
   - Right-hand side (after `~`) = independent variables (age, sex, worker class, education).
4. **data = dtrain** → tells R which dataset to use (training set).

Making Predictions with Linear Regression

- After building the model with lm(), use predict() function.
- Generates predictions for training and test sets.
- Predictions stored in new columns of data frames.
- Example :

dtest$predLogPINCP <- predict(model, newdata = dtest)

dtrain$predLogPINCP <- predict(model, newdata = dtrain)

Explanation:

Predictions for test set stored in dtest$predLogPINCP.

Predictions for training set stored in

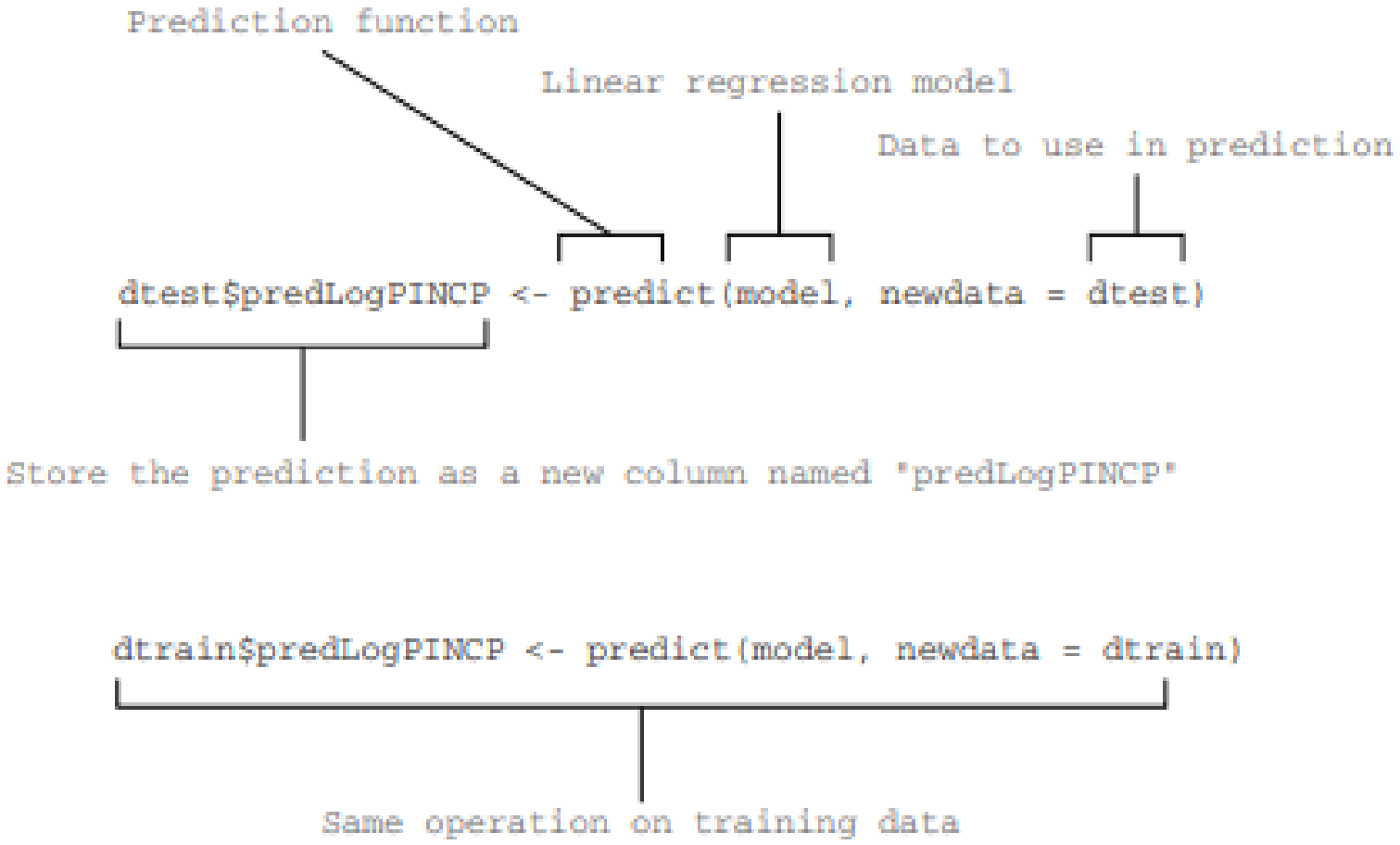


Figure 7.5 Making predictions with a linear regression model

Checking Predictions: True vs Predicted

- To check quality, compare:

  - Predicted values (X-axis) vs Actual values (Y-axis).

- If the model is good:

  - Points cluster near the line y = x (perfect prediction).

- In Listing 7.2, ggplot2 is used to plot:

  1. Points (predicted vs actual).

  2. Line of perfect prediction (dashed).

  3. Smooth trend line (to see if predictions are unbiased).

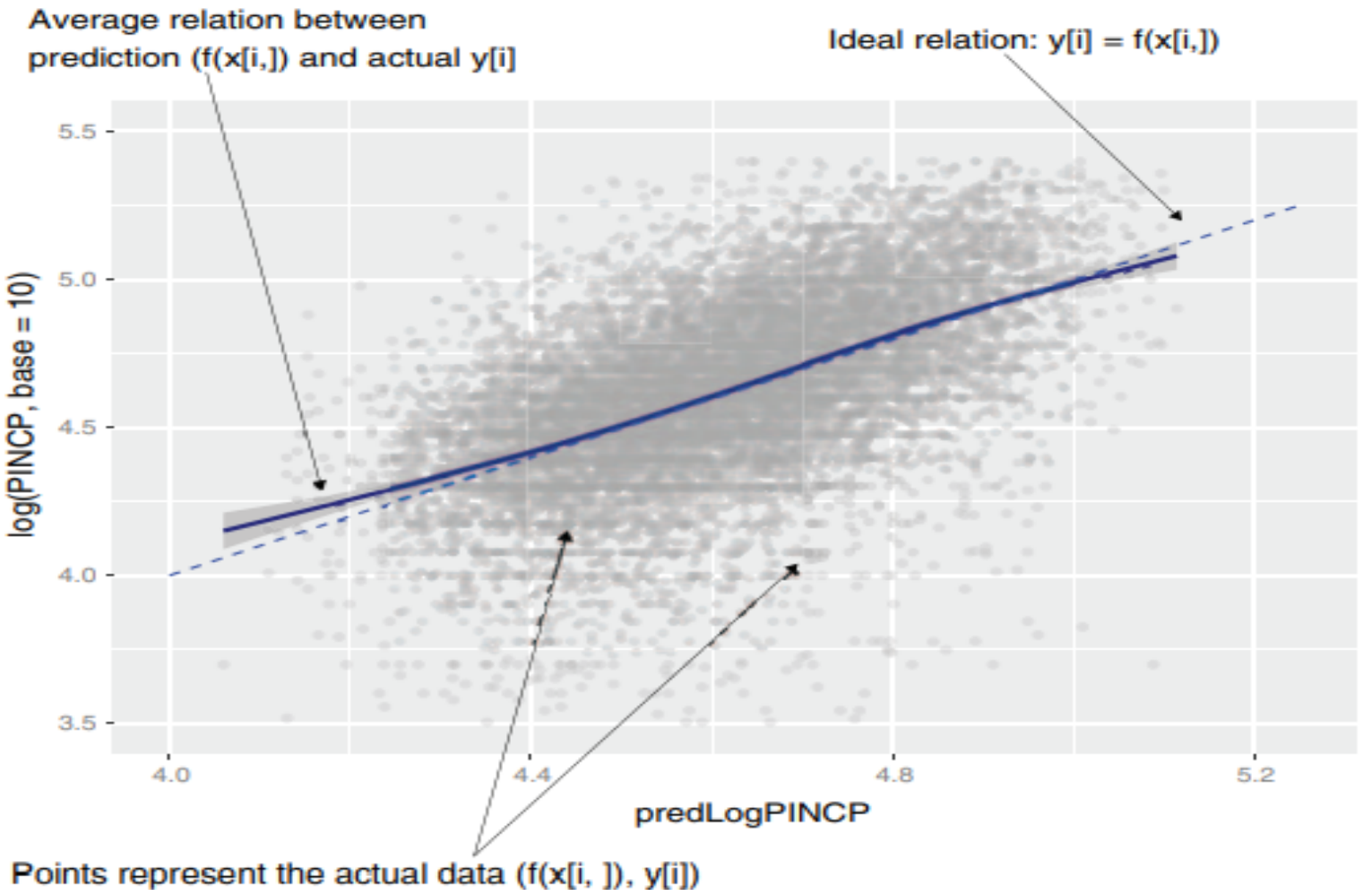
  Implemented with ggplot2 in R

Listing 7.2

Listing 7.2 Plotting log income as a function of predicted log income

```
library('ggplot2')
ggplot(data = dtest, aes(x = predLogPINCP, y = log10(PINCP))) +
   geom_point(alpha = 0.2, color = "darkgray") +
   geom_smooth(color = "darkblue") +
   geom_line(aes(x = log10(PINCP),                    <-- Plots the line x=y
                 y = log10(PINCP)),
             color = "blue", linetype = 2) +
   coord_cartesian(xlim = c(4, 5.25),                 <-- Limits the range of the
                   ylim = c(3.5, 5.5))                    graph for legibility
```

Figure 7.6 Plot of actual log income as a function of predicted log income



Cloud of points near line y = x.
Our model is “okay” but not perfect → points are spread out.

Residual Plot

- Another way to check model quality is using residuals:
  - Residual = Prediction – Actual.
- If model is unbiased → residuals should be scattered around 0 (no pattern).
- Example R code(7.3) :

```
ggplot(data = dtest, aes(x = predLogPINCP,
                         y = predLogPINCP - log10(PINCP))) +
  geom_point(alpha = 0.2, color = "darkgray") +
  geom_smooth(color = "darkblue") +
  ylab("residual error (prediction - actual)")
```

- - Patterns in residuals suggest poor fit.
- - Implemented with ggplot2 in R (Listing 7.3).

Figure 7.7 Plot of residual error as a function of prediction

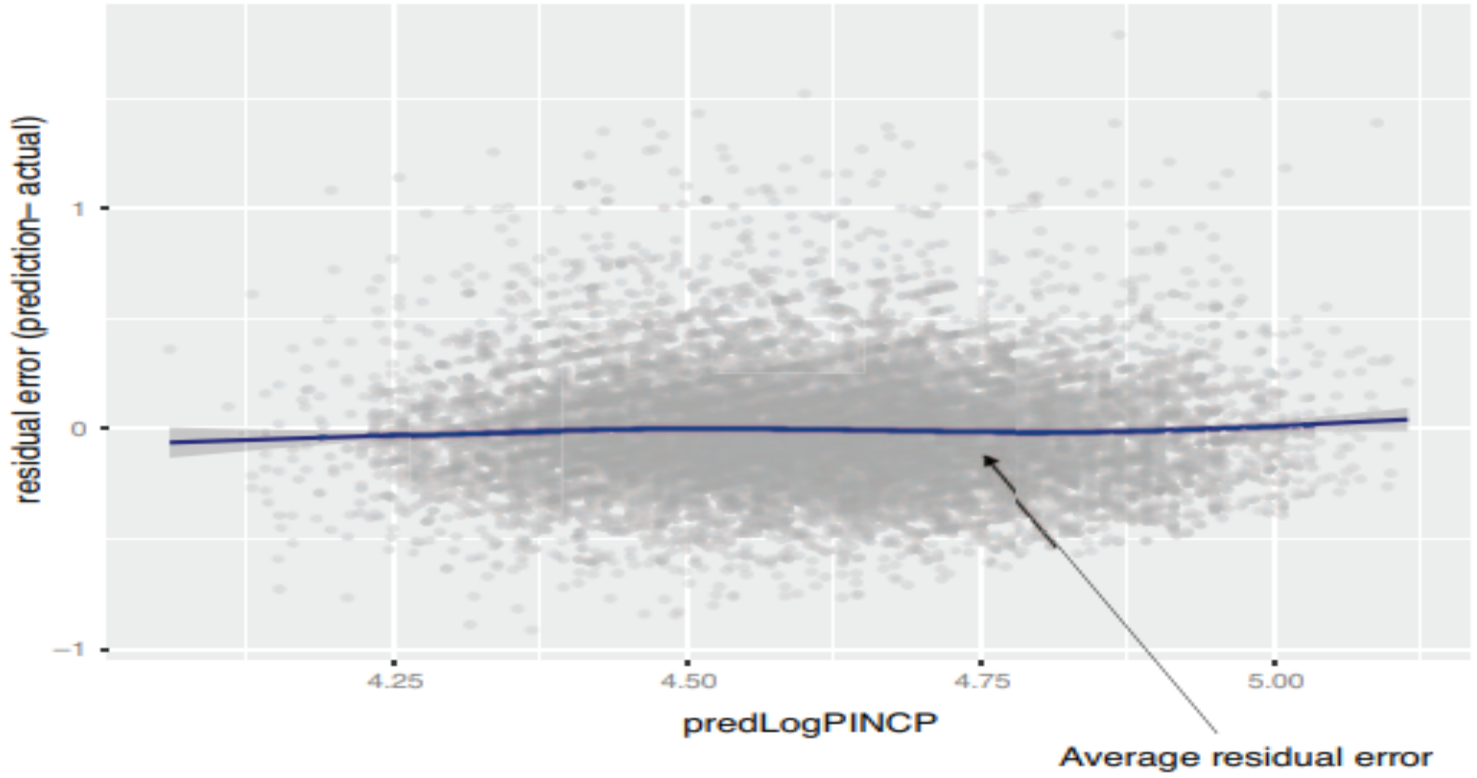


Figure 7.7 Plot of residual error as a function of prediction

Many points are far from 0 → possible low-quality fit.

Systematic Errors in Predictions

- If the smooth curve veers off the zero line, predictions are systematically wrong.
- Meaning:
  1. In some ranges, model overpredicts.
  2. In other ranges, it underpredicts.
- This happens when the data is not linear enough for linear regression.
- -Consider alternate models.

Figure 7.8 An example of systematic errors in model predictions

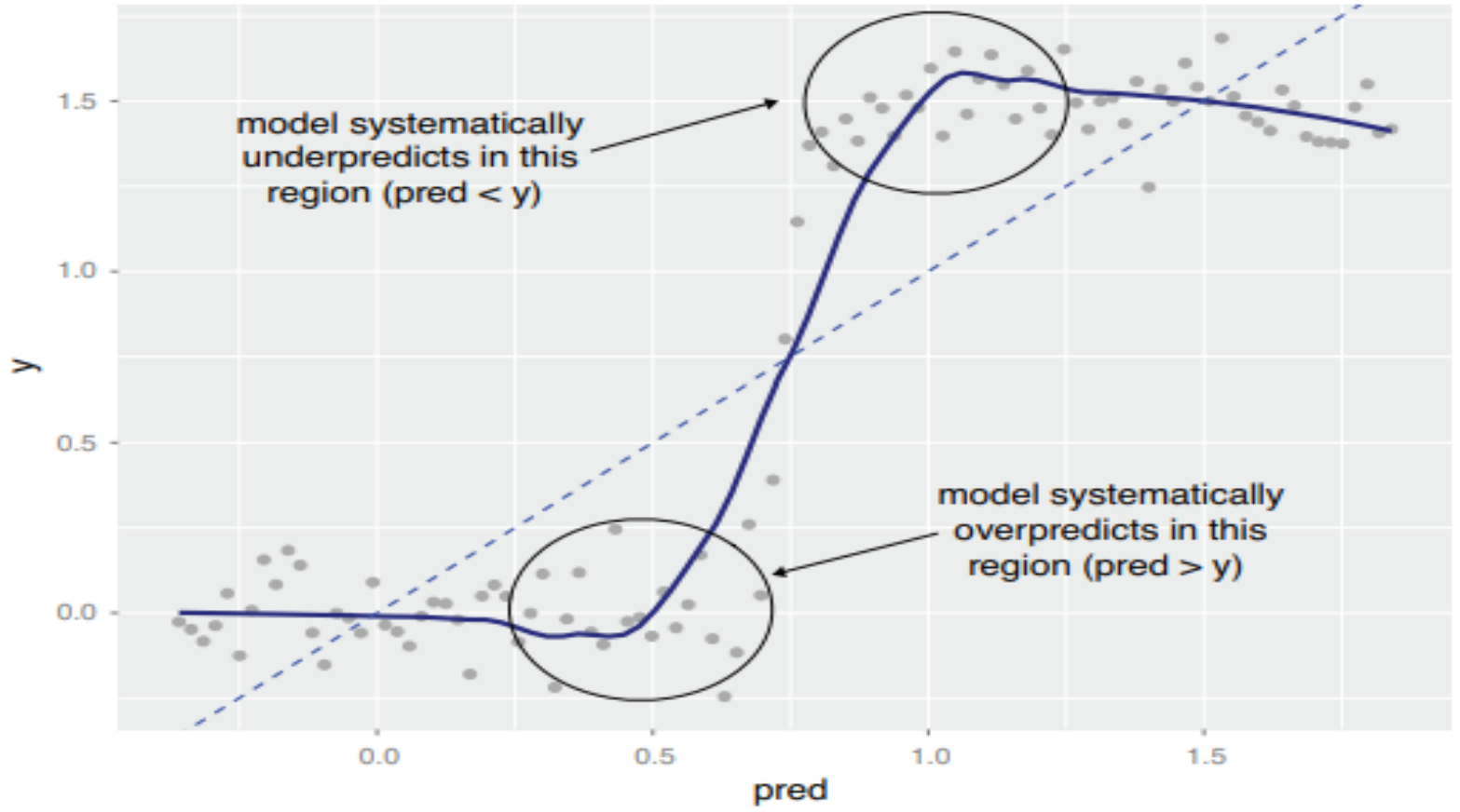


Figure 7.8 An example of systematic errors in model predictions

Evaluating Predictions: R-squared and RMSE

- - R-squared:
  - Fraction of variance explained by model
  - Ranges 0–1 (higher is better)
  - Example: ~0.3 → weak model

- - RMSE (Root Mean Square Error):
  - Average size of prediction error
  - Lower RMSE = better model

R-squared

- A quantitative way to measure fit.
- Formula in R:

```
rsq <- function(y, f) { 1 - sum((y - f)^2)/sum((y - mean(y))^2) }
rsq(log10(dtrain$PINCP), dtrain$predLogPINCP)
rsq(log10(dtest$PINCP), dtest$predLogPINCP)
```

- R-squared = fraction of variation in Y explained by the model.
- Ranges 0 to 1 (higher = better).
- Here: $R^2 \approx 0.3 \rightarrow$ weak model (not great, but not overfit).

RMSE – Root Mean Square Error

- Another measure of prediction error.
- Formula in R:

```
rmse <- function(y, f) { sqrt(mean( (y-f)^2 )) }
rmse(log10(dtrain$PINCP), dtrain$predLogPINCP)
rmse(log10(dtest$PINCP), dtest$predLogPINCP)
```

- RMSE = average prediction error size.
- Lower RMSE = better model.

Key Takeaways

- Use predict() to make predictions on train/test data.
- Check quality with plots:
  1. True vs Predicted (should be near y=x line).
  2. Residual Plot (errors should scatter randomly around 0).

- Watch for systematic errors (indicate poor model fit).
- Use R-squared to measure goodness of fit.
- Use RMSE to measure average prediction error.
- A good model = high R^2, low RMSE, and no systematic residual patterns.

Summary: Evaluating Model Quality

- - Use predict() to generate predictions.
- - Evaluate using plots:
- • True vs Predicted values
- • Residual plots
- - Detect systematic errors.
- - Measure fit with R-squared and RMSE.
- - A good model: high R^2, low RMSE, random residuals.